National University of Computer and Emerging Sciences

Lahore Campus

Computer Organization & Assembly Language (EE2003)

Sessional-II Exam

Total Time (Hrs): 1
Total Marks: 40
Total Questions: 2

Date: October 31st 2026

Course Instructor(s)

Dr. Aatira Anum, Ms. Samin Iftikhar, Ms. Aleena Ahmad, Ms. Sana Fatima, Mr. Salman Mubarak, Ms. Saira Arif, Ms. Shazia Ahmed

Roll No _____ Section _____ Student Signature _____

Do not write below this line

Instruction/Notes:

1. **This is an open book exam. Only textbook by "Belal Hashmi" is allowed. Sharing book or calculators is NOT ALLOWED. PAPERS/NOTES ATTACHED WITH BOOK ARE NOT ALLOWED.**
2. **Answer Q1 on the question paper and Q2 on the answer sheet.**
3. **In case of any ambiguity, make reasonable assumptions. Questions during exams are not allowed.**

CLO # 2: Describe the working of important x86 assembly primitives, including arithmetic, branching, bit manipulation, addressing modes and interrupt handling.

Q1: Short Questions [2+ 3 + 3 + 9 (includes 3 marks for working) + 3 = 20 marks]

(a) What would be the values of CX and SP after execution of the following code? The initial value of SP is 0xFFFE.

[org 0x0100] mov cx, 10 l1: push ax loop l1 mov ax, 0x4c00; terminate program int 0x21	**Show your working here.** CX=10 SP= 0x FFFC CX=9 SP= 0x FFFA CX=8 SP= 0x FFF8 CX=7 SP= 0x FFF6 CX=6 SP= ~~~ 4 CX=5 SP= 0x FFF2 CX=4 SP= 0x FFF0 CX=3 SP= 0x FFEE CX=2 SP= 0x FFEC CX=1 SP= 0x FFEA = 0xFFFE-20 = 0xFFEA	CX: 0 SP: 0x FFEA 2

(b) The following code does not terminate properly. Identify and correct the error. Do not rewrite the whole code. You cannot modify any instruction. Just mention the line number of the instruction to be removed or if you are adding an instruction, mention the instruction and line number where it should be added.

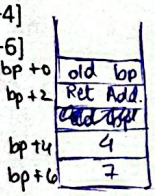
Scanned with CamScanner

1. [org 0x100] 2. mov ax, 15 3. call sum 4. mov bx, ax 5. mov ax, 0x4c00; terminate program 6. int 0x21 7. sum: push ax 8. add ax, 10 9. ret	I will insert an instruction between line 4 and line 5. pop ax ✓ 3

(c) Dry-run the code given in Left column and answer the questions in the right column.

[org 0x0100] start: mov ax, 7 push ax mov ax, 4 push ax call calculate jmp exit calculate: push bp mov bp, sp mov ax, [bp+4] mov bx, [bp+6] 4+7=11 ← add ax, bx 11x2=22 shl ax, 1 pop bp ret 4 exit: mov ax, 0x4C00 int 0x21	What is the final value of AX before the code exit? 22 What values are accessed by (1) [bp+6] 7 ? (2) [bp+ 4] 4 ? ✓ 3



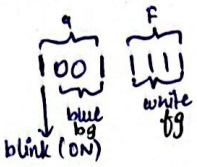


(d) Consider the code snippet in the left column and answer the questions in the rightmost column.

Scanned with CamScanner

National University of Computer and Emerging Sciences

Lahore Campus

mov ax, 0xB800 mov es, ax mov word [es:0x0190], 0x9F41 Attribute ↓ ↓ ASCII	**Show your working here.**  $41_{(16)}$ = A ⇒ Character = 29th ASCII character before ESC Physical Address = B8000 + 0190 = B8190 offset = 0x0190 = 400 Row = 400/160 = 2 Col = (400%160)/2 = 80/2 = 40 8	(a) Identify the character rendered on the monitor. ESC ✗ (b) Identify the foreground color. Bright white ✓ (c) Identify the background color. Blue ✓ (d) State whether blinking is active. Yes ✓ (e) Calculate the exact physical position (Row and Column) on the screen, assuming 0-indexed coordinates (Row 0–24, Col 0–79). Show Your Working. Row: 2 ✓ Column: 40 ✓

(e) Complete the following code such that it makes the data on 12th row blinking.

```
[org 0x0100]
; assume that screen is white on black initially, low intensity, no blinking
 mov ax, 0xb800
 mov es, ax
 mov ds, ax
 mov si, 160 * 12
 mov di, si
 mov cx, 80
 cld
loop1:
;Add your code here, use only string instructions to read and write the video memory data
        lodsw   ; AX = cell data
        xor ax, 0x8000   ; blink 0→1
        stosw   ; cell data = AX


loop loop1
mov ax, 0x4c00; terminate program
int 0x21
```

3

CLO # 3: Apply the knowledge of Intel x86 architecture to develop moderately complex and well-modularized assembly programs.

Q2: [20 marks] Write a subroutine that takes 3 parameters; source row number, number of rows to copy and destination row number. The subroutine should copy the rows starting from source row number of the

Scanned with CamScanner

display screen to the destination row number. The number of rows to copy are already provided in the parameters.

E.g., if the following parameters are provided
Source row number 1
Number of rows to copy 2
Destination row number 5, then row 1 and 2 of the display memory is copied to row 5 and 6.

You have to ensure that the source and destination row numbers are valid (between 0-24), and the number of rows to copy are actually available below the source or destination row number. If anything is invalid, then do nothing and return 1 from the subroutine else return 0.

E.g., if the source or destination row number is 24 and number of rows to copy are 4, then this is invalid as total number of rows are 25.

You need to deal with overlapping case as well. E.g., if source row number is 3, number of rows is 2 and destination row number is 4, then original row 3 and 4 of the display screen is copied to row 4 and 5.

You have to make sure that the code should work for all cases even if the destination row number is less than the source row number.

E.g., if the following parameters are provided
Source row number 5
Number of rows to copy 2
Destination row number 1, then row 5 and 6 of the display memory is copied to row 1 and 2.

The parameters must be passed and accessed through stack and no register values should be destroyed by the subroutine. Credit will be given for using String Instructions where required.

Scanned with CamScanner